Transactions and Concurrency Control – Terminology Concepts Reference

Database Systems

Based on Lecture 10 by Tiantian Liu

Fall 2025

Contents

1 Understanding Notation and Terminology

Plain English Guide - Understanding the Symbols

Throughout this cheatsheet, we use specific notation. Here's what everything means:
Common Symbols and Operations:

- **read(X, x)**: Read data item X from disk into variable x
- **Plain English:** Load the value of X into memory
- **write(X, x)**: Write variable x to data item X on disk
- **Plain English:** Save the value of x back to X on disk
- **T1, T2, T3**: Transaction identifiers
- **Plain English:** Names for different transactions (like "User A", "User B")
- **lock-S(X)**: Acquire shared lock on X
- **Plain English:** "I want to read X, others can read too"
- **lock-X(X)**: Acquire exclusive lock on X
- **Plain English:** "I want to write X, nobody else can access it"
- **unlock(X)**: Release lock on X
- **Plain English:** "I'm done with X, others can use it now"

Understanding Transactions:

- **Transaction**: A sequence of database operations that form a single logical unit
- **Plain English:** Like a business task (e.g., transfer money from A to B)
- **Schedule**: The order in which operations from multiple transactions execute
- **Plain English:** A timeline showing what happens when
- **Commit**: Transaction completes successfully, changes are permanent
- **Plain English:** "Save all changes for real"
- **Abort/Rollback**: Transaction fails, undo all changes
- **Plain English:** "Cancel everything, go back to the start"

Key Concept - Conflict:

- Two operations conflict if they access the same data item AND at least one is a write
- **Examples of conflicts:**
  - read(A) and write(A) – CONFLICT
  - write(A) and read(A) – CONFLICT
  - write(A) and write(A) – CONFLICT
  - read(A) and read(A) – NO CONFLICT (both just reading!)

2 Quick Reference Guide

Essential Concepts - At a Glance

ACID Properties (The Golden Rules):

- **Atomicity**: All or nothing - either complete transaction or none of it
- **Plain English:** Like a light switch - on or off, never halfway
- **Consistency**: Database goes from one valid state to another
- **Plain English:** Rules are never broken (e.g., money isn't created/destroyed)
- **Isolation**: Transactions don't interfere with each other
- **Plain English:** Each transaction thinks it's running alone
- **Durability**: Committed changes survive system crashes
- **Plain English:** Once saved, it stays saved forever

Schedule Types:

- **Serial Schedule**: Transactions run one after another (no overlap)
- **Plain English:** Like people using an ATM one at a time
- **Concurrent Schedule**: Operations from different transactions interleave
- **Plain English:** Like multitasking - switching between tasks
- **Serializable Schedule**: Concurrent but equivalent to some serial schedule
- **Plain English:** Looks like multitasking but gives same result as one-at-a-time

Conflict Serializability Test:

1. Draw precedence graph with transactions as nodes
2. **Plain English:** Make a diagram with circles for each transaction
3. Add edge Ti → Tj if Ti conflicts with Tj and Ti's operation comes first
4. **Plain English:** Draw arrows showing "who must go before whom"
5. If graph has NO cycles → Conflict serializable
6. **Plain English:** If you can't follow arrows in a loop, it's good!
7. If graph has cycles → NOT conflict serializable
8. **Plain English:** If arrows form a loop, there's a problem!

Two-Phase Locking (2PL):

- **Phase 1 - Growing**: Acquire locks, cannot release any
- **Plain English:** Collect all the keys you need, don't give any back yet
- **Phase 2 - Shrinking**: Release locks, cannot acquire any

- **Plain English:** Return keys, can't ask for more keys
- **Key Rule**: Once you release one lock, you can never acquire another
- **Guarantee**: 2PL ensures conflict serializability!

Lock Compatibility Matrix:

	S	**X**
S	Yes	No
X	No	No

- **Plain English:** Multiple readers OK, but writers need exclusive access

Complete Concurrency Issues Reference:

Problem	**Description**	**Example**	**Solution**
Lost Update	T2 overwrites T1's uncommitted changes	T1 reads A=100, T2 reads A=100, both write different values	Use exclusive locks before read-modify-write
Dirty Read	Reading uncommitted data from another transaction	T1 writes A=50, T2 reads A=50, T1 aborts → T2 has invalid data	Strict 2PL (hold X-locks until commit)
Non-Repeatable Read	Same query returns different values within transaction	T1 reads A=100, T2 updates A=200 and commits, T1 reads A=200	Hold S-locks until end of transaction
Phantom Read	New rows appear in repeated query	T1 counts 10 rows, T2 inserts row, T1 counts 11 rows	Predicate locking or table-level locks

3 Introduction: Why Transactions Matter

The Big Picture

What is a Transaction?

- **The Problem:** Databases have many users accessing data simultaneously
- **The Challenge:** How to ensure everyone's work is correct and doesn't interfere?
- **The Solution:** Transactions - logical units of work with guarantees

Why Do We Need Transactions?

- **Real-World Example:** Transferring $50 from Account A to Account B
- Step 1: Read balance of A
- Step 2: Subtract $50 from A
- Step 3: Write new balance to A

- Step 4: Read balance of B
- Step 5: Add $50 to B
- Step 6: Write new balance to B
- **What could go wrong?**
  - System crashes after Step 3 → $50 disappears! (Atomicity problem)
  - Another transaction reads A between Steps 3 and 6 → sees inconsistent state! (Isolation problem)
  - Power failure after Step 6 → changes must survive! (Durability problem)

The Two Main Challenges:

- **1. Failures**: Hardware crashes, power outages, software bugs
- **Plain English:** Things go wrong - how do we recover?
- **2. Concurrency**: Multiple users/transactions running simultaneously
- **Plain English:** Many people using the system at once - how do we keep them from interfering?

Key Insight:

- Transactions provide a simple programming model
- **Without transactions:** You worry about every possible failure and interference
- **With transactions:** Database handles it - you just say BEGIN and COMMIT
- **Analogy:** Like automatic transmission vs manual - database shifts the gears for you!

4 Transaction Concept

What is a Transaction?

Definition: A transaction is a unit of program execution that accesses and possibly updates various data items.
Plain English: A transaction is a sequence of operations that should be treated as a single, indivisible task.
Key Properties:

- Forms a single logical unit of work
- Contains one or more database operations
- Must preserve database consistency
- Either completes entirely or has no effect at all

Real-World Analogy:

- **Transaction = Shopping trip to grocery store**

- You make a shopping list (plan)
- Pick items from shelves (read operations)
- Put items in cart (local changes)
- Pay at checkout (commit) OR leave cart and exit (abort)
- **Key point:** Either you complete the entire shopping trip, or none of it counts!

4.1 ACID Properties

ACID Properties - The Foundation

To preserve data integrity, the database system must ensure ACID properties:
A - Atomicity (All or Nothing):

- Either all operations of the transaction are reflected in the database, or none are
- **Plain English:** Can't do half a transaction - it's like a light switch (on/off, not halfway)
- **Example:** Money transfer - can't debit A without crediting B
- **Implementation:** Achieved through logging and recovery mechanisms
- **What happens on failure?** System automatically undoes partial changes

C - Consistency (Rules Never Break):

- Execution of a transaction in isolation preserves database consistency
- **Plain English:** Transaction takes database from one valid state to another valid state
- **Example:** If sum(all accounts) = $1000 before, it must be $1000 after
- **Who ensures it?** Programmer writes correct transactions; DBMS enforces constraints
- **Key insight:** Consistency is a promise about the transaction logic

I - Isolation (Transactions Don't See Each Other):

- Each transaction must be unaware of other concurrently executing transactions
- **Plain English:** Even though many transactions run together, each one thinks it's alone
- **Example:** If T1 transfers money, T2 shouldn't see "money in flight"
- **Implementation:** Achieved through concurrency control (locks, etc.)
- **The illusion:** Serial execution, but with concurrent performance!

D - Durability (Changes are Permanent):

- After a transaction completes successfully, changes persist even if system crashes

- **Plain English:** Once you commit, changes are saved forever (like writing in permanent ink)
- **Example:** After "COMMIT" on money transfer, even if power fails, changes survive
- **Implementation:** Write-ahead logging to stable storage (disk)
- **Guarantee:** Committed data survives all failures (except disk destruction!)

ACID in Action - Money Transfer Example

Transaction: Transfer $50 from Account A to Account B
Initial State: A = $100, B = $200 (Total = $300)
Transaction Operations:

```
BEGIN TRANSACTION;
    read(A, a);           -- a = 100
    a := a - 50;          -- a = 50
    write(A, a);          -- A = 50
    read(B, b);           -- b = 200
    b := b + 50;          -- b = 250
    write(B, b);          -- B = 250
COMMIT;
```

Final State: A = $50, B = $250 (Total = $300)
ACID Guarantees:

- **Atomicity:** If crash occurs after write(A) but before write(B), system rolls back write(A). We never have A=$50, B=$200 (money disappeared!)
- **Consistency:** Total money before = Total money after = $300 (invariant preserved)
- **Isolation:** If another transaction reads A and B during transfer, it either sees (100,200) or (50,250), never (50,200)
- **Durability:** After COMMIT, even if power fails immediately, A=$50 and B=$250 persist

4.2 Transaction Operations

Transaction Control Statements

BEGIN TRANSACTION:

- Marks the start of a transaction
- **SQL Syntax:** `BEGIN;` or `START TRANSACTION;`
- **Plain English:** "I'm starting a new task"
- All following statements are part of this transaction until COMMIT or ROLLBACK

COMMIT:

- Marks successful end of transaction

- **SQL Syntax:** `COMMIT;`
- **Plain English:** "Save all my changes permanently"
- Makes all changes visible to other transactions
- Changes become durable (survive crashes)

ROLLBACK (or ABORT):

- Causes transaction to abort
- **SQL Syntax:** `ROLLBACK;`
- **Plain English:** "Cancel everything, undo all changes"
- All changes are discarded
- Database returns to state before BEGIN

SAVEPOINT:

- Defines a point within a transaction
- **SQL Syntax:** `SAVEPOINT savepoint_name;`
- **Plain English:** "Create a checkpoint I can return to"
- Allows partial rollback to the savepoint
- **Rollback to savepoint:** `ROLLBACK TO savepoint_name;`

Using Savepoints

Scenario: Inserting multiple records, want to rollback some but not all

```
BEGIN;
INSERT INTO tab VALUES (1, 'Alice');
SAVEPOINT A;
INSERT INTO tab VALUES (2, 'Bob');
SAVEPOINT B;
INSERT INTO tab VALUES (3, 'Charlie');
SELECT * FROM tab;              -- Shows: Alice, Bob, Charlie
ROLLBACK TO A;                  -- Undo inserts after savepoint A
SELECT * FROM tab;              -- Shows: Alice (Bob and Charlie removed)
INSERT INTO tab VALUES (4, 'David');
COMMIT;                         -- Final result: Alice, David
```

Plain English Explanation:

- Started with Alice
- Created checkpoint A
- Added Bob and Charlie
- Changed mind, rolled back to checkpoint A (removed Bob and Charlie)

- Added David instead
- Committed final changes (Alice and David saved)

5 Concurrent Executions and Schedules

Why Concurrency Matters

The Problem:

- Multiple transactions submitted by different users simultaneously
- **Example:** Bank has thousands of ATM transactions per second
- Could execute transactions serially (one at a time), but this is SLOW
- **Analogy:** Like having only one checkout lane at a supermarket

The Solution - Concurrent Execution:

- Allow operations from different transactions to interleave
- **Analogy:** Multiple checkout lanes, plus express lanes
- **Benefits:**
  - Increased throughput (more transactions per second)
  - Reduced waiting time (short transactions don't wait behind long ones)
  - Better resource utilization (CPU works while disk I/O happens)

The Challenge:

- Concurrency can lead to problems if not managed carefully
- Need to ensure concurrent execution is "correct"
- **Definition of correct:** Result should be equivalent to some serial execution

5.1 Schedules

What is a Schedule?

Definition: A schedule is a sequence of instructions that specifies the chronological order in which operations of concurrent transactions are executed.
Plain English: A schedule is a timeline showing when each operation from each transaction happens.
Components:

- Operations from multiple transactions
- Order in which they execute
- Must preserve order of operations within each transaction
- **Key constraint:** If operation A comes before operation B in transaction T, then A must come before B in any schedule containing T

Types of Schedules

Serial Schedule:

- Operations of transactions execute sequentially with no overlap
- **Plain English:** Run T1 completely, then T2 completely, then T3, etc.
- **Example:** T1 → T2 → T3 (all of T1, then all of T2, then all of T3)
- **Guarantee:** Always produces correct result (by definition!)
- **Drawback:** Poor performance (no concurrency benefits)

Concurrent Schedule:

- Operations from different transactions interleave
- **Plain English:** Switch back and forth between transactions
- **Example:** T1's read, then T2's read, then T1's write, then T2's write
- **Benefit:** Better performance
- **Risk:** Might produce incorrect result if not carefully controlled

Serializable Schedule:

- A concurrent schedule that is equivalent to some serial schedule
- **Plain English:** Looks like multitasking, but gives the same answer as serial execution
- **The Holy Grail:** Performance of concurrency + correctness of serial execution
- **Goal of concurrency control:** Ensure all concurrent schedules are serializable

Schedule Examples - Money Transfer

Transactions:

- T1: Transfer $50 from A to B

- T2: Transfer 10% interest from A to B

Initial State: A = \$1000, B = \$2000
Schedule 1 (Serial: T1 then T2):

```
T1: read(A)              -- A = 1000
T1: A := A - 50          -- A = 950
T1: write(A)             -- A = 950
T1: read(B)              -- B = 2000
T1: B := B + 50          -- B = 2050
T1: write(B)             -- B = 2050
T2: read(A)              -- A = 950
T2: A := A * 0.9         -- A = 855
T2: write(A)             -- A = 855
T2: read(B)              -- B = 2050
T2: B := B + A*0.1       -- B = 2050 + 95 = 2145
T2: write(B)             -- B = 2145
```

Final State: A = \$855, B = \$2145 (Total = \$3000)
Schedule 2 (Serial: T2 then T1):

```
T2: read(A)              -- A = 1000
T2: A := A * 0.9         -- A = 900
T2: write(A)             -- A = 900
T2: read(B)              -- B = 2000
T2: B := B + A*0.1       -- B = 2000 + 100 = 2100
T2: write(B)             -- B = 2100
T1: read(A)              -- A = 900
T1: A := A - 50          -- A = 850
T1: write(A)             -- A = 850
T1: read(B)              -- B = 2100
T1: B := B + 50          -- B = 2150
T1: write(B)             -- B = 2150
```

Final State: A = \$850, B = \$2150 (Total = \$3000)
Note: Different serial schedules give different final states, but both are correct!

5.2 Problems with Concurrency

Problem 1: Lost Update

What is it? One transaction's update is overwritten by another transaction.
Plain English: Two people editing the same document; one person's changes get lost.
Example Schedule:

```
Step    T1                      T2
1       read(A, a1)             -- a1 = 1000
2       a1 := a1 - 300          -- a1 = 700
3                               read(A, a2)       -- a2 = 1000 (old value!)
4                               a2 := a2 * 1.03   -- a2 = 1030
5                               write(A, a2)      -- A = 1030
6       write(A, a1)            -- A = 700 (overwrites T2's update!)
```

Problem: T2's interest calculation is lost! Should be: 1000 → 1030 (interest) → 730 (withdrawal)
Actual result: 1000 → 700 (only withdrawal applied)
Root cause: T2 wrote after T1 read but before T1 wrote; T1's write overwrote T2's change

Problem 2: Dirty Read (Reading Uncommitted Data)

What is it? A transaction reads data written by another transaction that hasn't committed yet.
Plain English: Reading someone's draft before they finalize it; if they cancel, you read wrong data.
Example Schedule:

```
Step    T1                      T2
1       read(A, a1)             -- a1 = 1000
2       a1 := a1 - 300          -- a1 = 700
3       write(A, a1)            -- A = 700
4                               read(A, a2)      -- a2 = 700 (dirty read!)
5                               a2 := a2 * 1.03  -- a2 = 721
6                               write(A, a2)     -- A = 721
7       read(B, b1)
8       ...
9       ABORT                   -- T1 rolls back! A should return to 1000
```

Problem: T2 read A = 700, but T1 aborted, so A should never have been 700!
Result: T2 made a decision (applied interest) based on data that "never happened"
Root cause: T2 read uncommitted data from T1

Problem 3: Non-Repeatable Read

What is it? A transaction reads the same data twice and gets different values.
Plain English: You check your account balance twice in one session, and it changes!
Example Schedule:

```
T1                                  T2
SELECT sum(balance) FROM account;   -- Result: $10,000
                                    UPDATE account SET balance = 42000
                                    WHERE accountID = 123;
                                    COMMIT;
SELECT sum(balance) FROM account;   -- Result: $52,000 (different!)
```

Problem: T1 reads total balance twice, gets different answers
Impact: Reports become inconsistent; can't trust that data stays stable during transaction
Root cause: Other transactions modified data between T1's reads

Problem 4: Phantom Read

What is it? A transaction re-executes a query and finds new rows that weren't there before.
Plain English: You count people in a room twice; second time there are new people!

Example Schedule:

```
T1                                    T2
SELECT sum(balance) FROM account;     -- Result: $10,000 (10 accounts)
                                      INSERT INTO account
                                      VALUES (C, 1000, ...);
                                      COMMIT;
SELECT sum(balance) FROM account;     -- Result: $11,000 (11 accounts!)
```

Problem: T1's second query sees a "phantom" row (new account C)
Difference from non-repeatable read: Here, NEW rows appear (not existing rows changing)
Root cause: Other transactions inserted/deleted rows matching T1's query

6 Serializability

Understanding Serializability - The Core Idea

The Central Question: When is a concurrent schedule "correct"?
Basic Assumption: Each transaction, when run in isolation, preserves database consistency.

- **Plain English:** If we run one transaction alone on a consistent database, it produces a consistent database
- **Example:** Money transfer preserves total money in system

Serial Execution:

- Run transactions one at a time (T1 completely, then T2 completely, etc.)
- **Guarantee:** If each transaction preserves consistency, serial execution preserves consistency
- **Plain English:** If each step is safe, doing them one at a time is safe

Serializability:

- A concurrent schedule is serializable if it produces the same result as SOME serial execution
- **Plain English:** It's OK to interleave operations, as long as the final result matches some serial order
- **Key insight:** Don't care which serial order, just that there exists one
- **Guarantee:** Serializable schedules preserve consistency!

Conflict Equivalence

Definition: Two schedules are conflict equivalent if one can be transformed into the other by swapping non-conflicting consecutive operations.
Conflict vs Non-Conflict Pairs:

Non-Conflict Pairs (can be swapped):

- R(A) and R(A) – Both reading same item (read-read is safe)
- R(B) and R(A) – Different data items
- W(B) and R(A) – Different data items
- R(B) and W(A) – Different data items
- W(A) and W(B) – Different data items

Conflict Pairs (CANNOT be swapped):

- R(A) and W(A) – Same item, one is write
- W(A) and R(A) – Same item, one is write
- W(A) and W(A) – Same item, both writes

Key Rule: Operations conflict if and only if they access the **same data item** AND **at least one is a write**.
Plain English:

- If operations touch different items → No conflict (order doesn't matter)
- If both operations just read → No conflict (readers don't interfere)
- If same item AND at least one write → Conflict! (order matters)

6.1 Conflict Serializability

Conflicting Operations

Definition: Two operations conflict if:

1. They belong to different transactions
2. They access the same data item (e.g., both access X)
3. At least one of them is a write operation

Conflict Matrix:

Op1 / Op2	**read(X)**	**write(X)**
read(X)	No conflict	**CONFLICT**
write(X)	**CONFLICT**	**CONFLICT**

Plain English Explanation:

- **read-read:** No conflict (both just looking, don't interfere)
- **read-write:** Conflict! (one reading while other modifying)
- **write-read:** Conflict! (one modifying while other reading)
- **write-write:** Conflict! (both trying to modify - order matters!)

Why Conflicts Matter:

- Non-conflicting operations can be swapped without changing result
- **Example:** T1 reads X, T2 reads Y → order doesn't matter
- Conflicting operations CANNOT be swapped (order matters!)
- **Example:** T1 writes X, T2 reads X → swapping changes what T2 reads

Identifying Conflicts

Example 1: Conflicts on same data item

```
T1: read(X)      T2: write(X)     → CONFLICT (read-write on X)
T1: write(X)     T2: read(X)      → CONFLICT (write-read on X)
T1: write(X)     T2: write(X)     → CONFLICT (write-write on X)
T1: read(X)      T2: read(X)      → NO CONFLICT (read-read on X)
```

Example 2: No conflicts on different data items

```
T1: read(X)      T2: write(Y)     → NO CONFLICT (different items!)
T1: write(X)     T2: read(Y)      → NO CONFLICT (different items!)
T1: write(X)     T2: write(Y)     → NO CONFLICT (different items!)
```

Key Rule: Must access SAME data item AND at least one write for conflict!

Conflict Serializability

Definition: A schedule S is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting consecutive operations.
Plain English: Can we rearrange operations (only swapping ones that don't conflict) to get a serial schedule?
Process:

1. Start with schedule S
2. Find consecutive operations that don't conflict
3. Swap them
4. Repeat until you get a serial schedule (or determine you can't)

If successful: S is conflict equivalent to that serial schedule
If impossible: S is NOT conflict serializable

Testing Conflict Serializability by Swapping

Given Schedule:

```
T1: read(A)
T1: write(A)
T2: read(A)
T2: write(A)
T1: read(B)
T1: write(B)
```

Goal: Transform to serial schedule (either T1→T2 or T2→T1)

Can we swap operations?

- T1:write(A) and T2:read(A)? NO - they conflict (write-read on A)
- T2:write(A) and T1:read(B)? YES - different data items, can swap!

After swapping T2:write(A) with T1:read(B):

```
T1: read(A)
T1: write(A)
T2: read(A)
T1: read(B)        ← swapped
T2: write(A)       ← swapped
T1: write(B)
```

Continue swapping T2:read(A) with T1:read(B):

```
T1: read(A)
T1: write(A)
T1: read(B)        ← swapped
T2: read(A)        ← swapped
T2: write(A)
T1: write(B)
```

Can we get all of T1 before T2? Let's see if we can swap T2:read(A) with T1:write(B)...

- These operations don't conflict (different data items)
- Can swap!

Final serial schedule (T1 → T2):

```
T1: read(A)
T1: write(A)
T1: read(B)
T1: write(B)
T2: read(A)
T2: write(A)
```

Conclusion: Schedule is conflict serializable!

6.2 Precedence Graph (Conflict Graph)

Precedence Graph - The Easy Way

Why? Swapping operations manually is tedious for large schedules!
Better Method: Draw a precedence graph (also called conflict graph)
How to construct:

1. Create a node for each transaction
2. **Plain English:** Draw a circle for T1, T2, T3, etc.
3. For each pair of conflicting operations (Ti's op before Tj's op):

4. Draw an edge from Ti → Tj
5. **Plain English:** Draw arrow from Ti to Tj (Ti must come before Tj)
6. Optionally label edge with the data item (e.g., "A")

The Test:

- **If graph has NO cycles** → Schedule IS conflict serializable
- **Plain English:** If you can't follow arrows in a loop, it's good
- **If graph has cycles** → Schedule is NOT conflict serializable
- **Plain English:** If arrows form a loop, there's a problem

Why cycles are bad:

- Cycle T1 → T2 → T1 means: T1 must come before T2 AND T2 must come before T1
- **Plain English:** Impossible! Can't have T1 both before and after T2
- No serial schedule can satisfy these constraints

If no cycles - Finding serial order:

- Perform topological sort on the graph
- **Plain English:** Find an order where all arrows point forward
- This gives you an equivalent serial schedule!

Precedence Graph Example 1 - Serializable

Schedule:

```
T1: read(A)
T1: write(A)
T2: read(B)
T2: write(B)
T1: read(B)
T1: write(B)
T3: read(B)
T3: write(B)
T3: read(A)
T3: write(A)
```

Find conflicts:

- T1 writes A before T3 reads A → Edge: T1 → T3 (labeled A)
- T2 writes B before T1 reads B → Edge: T2 → T1 (labeled B)
- T1 writes B before T3 reads B → Edge: T1 → T3 (labeled B)

Precedence Graph:

```
T2 ----→ T1 ----→ T3
   (B)      (A,B)
```

Analysis:

- No cycles! Path is T2 → T1 → T3
- Schedule IS conflict serializable
- Equivalent serial order: T2, T1, T3

Precedence Graph Example 2 - NOT Serializable

Schedule:

```
T1: read(A)
T1: write(A)
T2: read(B)
T2: write(B)
T3: read(B)
T3: write(B)
T1: read(B)
T1: write(B)
T2: read(A)
T2: write(A)
```

Find conflicts:

- T1 writes A before T2 reads A → Edge: T1 → T2
- T2 writes B before T3 reads B → Edge: T2 → T3
- T3 writes B before T1 reads B → Edge: T3 → T1

Precedence Graph:

```
T1 ----→ T2
 ↑        ↓
 |        |
  T3 ←-
```

Analysis:

- CYCLE detected: T1 → T2 → T3 → T1
- Schedule is NOT conflict serializable
- Impossible to find equivalent serial schedule
- **Plain English:** T1 must come before T2, T2 before T3, and T3 before T1 - impossible!

7 Concurrency Control: Lock-Based Protocols

Why Locking?

The Problem: How do we ensure schedules are serializable?
Solution: Use locks to control access to data items
The Idea:

- Before accessing data, transaction must acquire a lock
- **Plain English:** Like checking out a library book - only one person can have it
- Lock prevents other transactions from interfering
- When done, transaction releases the lock

Two Types of Locks:

- **Shared (S) lock:** For reading
- **Plain English:** "I want to look at this, others can look too"
- **Exclusive (X) lock:** For writing
- **Plain English:** "I want to modify this, nobody else can touch it"

Real-World Analogy:

- **Shared lock = Library book:** Multiple people can read the same book
- **Exclusive lock = Editing a document:** Only one person can edit at a time

7.1 Lock Types and Operations

Lock Operations

lock-S(Q): Shared Lock

- Grants shared access to data item Q
- **Plain English:** "I want to READ Q"
- Multiple transactions can hold shared locks on Q simultaneously
- **Use case:** SELECT queries (just reading data)

lock-X(Q): Exclusive Lock

- Grants exclusive access to data item Q
- **Plain English:** "I want to WRITE Q"
- Only ONE transaction can hold exclusive lock on Q
- No other locks (shared or exclusive) allowed on Q
- **Use case:** UPDATE, INSERT, DELETE queries (modifying data)

unlock(Q): Release Lock

- Releases the lock on data item Q
- **Plain English:** "I'm done with Q, others can use it"
- Makes Q available for other transactions

Lock Compatibility Matrix

The Rules: Can transaction T2 acquire a lock if T1 already holds a lock?

T1 holds / T2 requests	Shared (S)	Exclusive (X)
Shared (S)	Yes	No (must wait)
Exclusive (X)	No (must wait)	No (must wait)

Plain English Rules:

- **S + S = OK:** Multiple readers allowed
- **S + X = WAIT:** Can't write while someone is reading
- **X + S = WAIT:** Can't read while someone is writing
- **X + X = WAIT:** Can't have two writers

Key Principle:

- **Readers don't block readers** (S + S compatible)
- **Writers block everyone** (X incompatible with everything)

Simple Locking Example

Transaction T2:

```
lock-S(A);
read(A);
unlock(A);
lock-S(B);
read(B);
unlock(B);
display(A + B);
```

Plain English:

- Lock A for reading
- Read A
- Release lock on A
- Lock B for reading
- Read B
- Release lock on B
- Display sum

Warning: This locking pattern is NOT sufficient to guarantee serializability!
Why not? Unlocking too early allows other transactions to interfere. We need Two-Phase Locking (coming next)!

7.2 Problems with Simple Locking

Problem: Early Unlocking Causes Non-Serializability

Scenario:

- T1: Transfer $50 from B to A
- T2: Display total of A and B
- Initial: A = $100, B = $200

Schedule with Early Unlocking:

```
T1                          T2
lock-X(B)
read(B)                     -- B = 200
B := B - 50                 -- B = 150
write(B)                    -- B = 150
unlock(B)                   -- ← Released lock too early!
                            lock-S(A)
                            read(A)                 -- A = 100
                            unlock(A)
                            lock-S(B)
                            read(B)                 -- B = 150
                            unlock(B)
                            display(A + B)          -- Shows 250
lock-X(A)
read(A)                     -- A = 100
A := A + 50                 -- A = 150
write(A)                    -- A = 150
unlock(A)
```

Problem:

- T2 displays $250 (A=100 + B=150)
- But this is an inconsistent state!
- After T1 completes: A = $150, B = $150 (Total = $300)
- T2 saw money "in flight" - only saw debit from B, not credit to A
- This doesn't match ANY serial execution (would show either $300 or $300)

Root Cause: T1 released lock on B before completing entire transaction
Solution: Two-Phase Locking (hold all locks until transaction completes)

8 Two-Phase Locking (2PL)

Two-Phase Locking Protocol

The Golden Rule: A protocol that ensures conflict serializability
Two Phases:
Phase 1: Growing Phase

- Transaction may acquire locks (lock-S, lock-X)
- Transaction may NOT release any locks
- **Plain English:** Collect all the keys you need, don't give any back yet

Phase 2: Shrinking Phase

- Transaction may release locks (unlock)
- Transaction may NOT acquire any new locks
- **Plain English:** Return keys, can't ask for more

The Critical Rule:

- Once a transaction releases ANY lock, it can NEVER acquire another lock
- **Plain English:** Once you start giving back keys, you can't ask for more keys
- The point where first lock is released is called the "lock point"

Guarantee:

- **If all transactions follow 2PL, all schedules are conflict serializable!**
- This is the fundamental theorem of 2PL

Identifying 2PL: Example 1 - YES

Transaction Ti:

```
lock-X(A)       ← Growing phase starts
lock-X(B)       ← Still growing
lock-X(C)       ← Still growing
unlock(A)       ← Shrinking phase starts (lock point!)
unlock(B)       ← Still shrinking
unlock(C)       ← Still shrinking
```

Analysis:

- Growing phase: Acquires locks on A, B, C
- Shrinking phase: Releases locks on A, B, C
- No lock acquired after first release
- **Follows 2PL: YES**

Identifying 2PL: Example 2 - NO

Transaction Ti:

```
lock-X(A)       ← Growing phase
lock-X(B)       ← Still growing
unlock(B)       ← Shrinking phase starts
unlock(A)       ← Still shrinking
lock-X(C)       ← Oops! Acquiring lock during shrinking phase!
unlock(C)
```

Analysis:

- Released unlock(B) at step 3
- Then acquired lock-X(C) at step 5
- **Violation:** Acquired lock after releasing a lock
- **Follows 2PL: NO**

2PL Fixing the Early Unlock Problem

Correct Schedule with 2PL:

```
T1                          T2
lock-X(B)
read(B)
B := B - 50
write(B)
lock-X(A)                   ← Acquire all locks before releasing any
read(A)
A := A + 50
write(A)
unlock(B)                   ← Now release locks
unlock(A)
                            lock-S(A)                ← Must wait for T1 to finish
                            lock-S(B)
                            read(A)                  -- A = 150
                            read(B)                  -- B = 150
                            unlock(A)
                            unlock(B)
                            display(A + B)           -- Shows 300
```

Result:

- T2 must wait for T1 to complete
- T2 sees consistent state (A = 150, B = 150, total = 300)
- Schedule is serializable (equivalent to serial schedule T1 → T2)
- **2PL prevents the inconsistency!**

8.1 Strict Two-Phase Locking

Strict 2PL - A Stronger Variant

Definition: A transaction must hold all exclusive (X) locks until it commits or aborts.
Difference from Basic 2PL:

- Basic 2PL: Can release locks after lock point, before commit
- Strict 2PL: Must hold X-locks until commit/abort

Benefits:

- **Prevents cascading rollbacks**
- If T1 aborts, don't need to abort T2 (which read T1's data)
- **Simpler recovery**
- No transaction sees uncommitted data

Usage:

- Most commercial databases use Strict 2PL (not just basic 2PL)
- **Trade-off:** Longer lock hold times, but simpler and safer

9 Deadlock

Understanding Deadlock

What is Deadlock?

- A situation where transactions are waiting for each other in a cycle
- **Plain English:** "You have what I need, I have what you need, we're both stuck!"
- Nobody can proceed

Classic Example - The Dining Philosophers:

- 5 philosophers sitting at a round table
- 5 chopsticks (one between each pair)
- Each philosopher needs 2 chopsticks to eat
- If everyone picks up left chopstick simultaneously, nobody can eat!
- **Deadlock:** Everyone waiting for the person on their right to release chopstick

Database Example:

- T1 locks A, waits for B
- T2 locks B, waits for A
- **Deadlock:** T1 and T2 wait for each other forever!

9.1 Deadlock Detection

Wait-For Graph

Purpose: Detect deadlocks in the system
How to construct:

1. **Vertices (nodes):** Represent transactions
2. **Plain English:** Draw a circle for each transaction
3. **Edges (arrows):** Ti → Tj if Ti is waiting for a lock held by Tj
4. **Plain English:** Arrow means "Ti is waiting for Tj to finish"

Deadlock Detection Rule:

- **If wait-for graph has a CYCLE** → Deadlock exists!
- **Plain English:** If you can follow arrows in a loop, transactions are stuck
- **If wait-for graph has NO cycles** → No deadlock

What to do when deadlock detected:

- Choose a "victim" transaction
- Rollback (abort) the victim
- This breaks the cycle, allowing others to proceed
- Victim transaction can be restarted later

Wait-For Graph Example 1 - Deadlock!

Scenario:

```
T1                  T2                  T3                  T4
lock-X(A)
                    lock-X(B)
                                        lock-X(C)
                                                            lock-X(B) -- waits for T2
                    lock-X(D)
                                        lock-X(E)
                                        lock-X(D) -- waits for T2
lock-X(E) -- waits for T3
                                                            lock-X(A) -- waits for T1
```

Who is waiting for whom?

- T4 waits for T2 (wants B, held by T2)
- T3 waits for T2 (wants D, held by T2)
- T1 waits for T3 (wants E, held by T3)
- T4 waits for T1 (wants A, held by T1)

Wait-For Graph:

```
T1 ←---- T4
 ↓        ↑
 |        |
T3 ---→ T2
```

Analysis:

- CYCLE detected: T1 → T3 → T2 → T4 → T1
- **Deadlock exists!**
- **Solution:** Abort one transaction (e.g., T4) to break cycle

Wait-For Graph Example 2 - No Deadlock

Scenario:

```
T1                  T2                  T3
lock-X(A)
                    lock-X(B)
                                        lock-X(C)
lock-X(A) -- already has it
                    lock-X(B) -- already has it
                                        lock-X(D)
```

Who is waiting for whom?

- T1: Not waiting (has all locks it needs)
- T2: Not waiting (has all locks it needs)
- T3: Not waiting (has all locks it needs)

Wait-For Graph:

```
T1      T2      T3
(no edges - nobody is waiting)
```

Analysis:

- No cycles (in fact, no edges at all!)
- **No deadlock**

Wait-For Graph Practice Exercise

Scenario:

```
T1                  T2                  T3
lock-X(A)
                    lock-X(B)
                                        lock-X(C)
lock-X(A) -- has it
                    lock-X(B) -- has it
lock-X(D)
```

Draw the wait-for graph and determine if there is a deadlock.
Solution:
Who is waiting for whom?

- T1 is NOT waiting (has A, got D)
- T2 is NOT waiting (has B)
- T3 is NOT waiting (has C)

Wait-For Graph:

```
T1      T2      T3

No edges!
```

Analysis:

- No edges in graph
- No cycles
- **No deadlock**

9.2 Deadlock Recovery

Handling Deadlocks

When deadlock is detected:
Step 1: Select a Victim

- Choose which transaction to abort
- **Factors to consider:**
  - How much work has transaction done? (prefer to abort one with less work)
  - How many transactions will be affected?
  - How long has transaction been waiting?

Step 2: Rollback

- **Total rollback:** Abort entire transaction, restart from beginning
- **Plain English:** Cancel everything, start over
- **Partial rollback:** Rollback only far enough to break deadlock
- **Plain English:** Undo just enough to release needed locks
- Most systems use total rollback (simpler)

Step 3: Prevent Starvation

- Same transaction shouldn't always be chosen as victim
- **Solution:** Track how many times transaction has been aborted
- Give priority to transactions that have been restarted multiple times

10 Summary

Key Concepts Review

Transactions:

- Logical unit of work with ACID properties
- Atomicity: All or nothing
- Consistency: Valid state to valid state
- Isolation: Transactions don't interfere
- Durability: Committed changes survive crashes

Schedules:

- Serial: Transactions run one at a time (safe but slow)
- Concurrent: Operations interleave (fast but risky)
- Serializable: Concurrent but equivalent to serial (best of both!)

Conflict Serializability:

- Two operations conflict if: different transactions, same data item, at least one write
- Test: Draw precedence graph
- No cycles → Serializable
- Has cycles → Not serializable

Locking:

- Shared (S) locks: For reading (many allowed)
- Exclusive (X) locks: For writing (only one allowed)
- Lock compatibility: S+S ok, everything else waits

Two-Phase Locking (2PL):

- Growing phase: Acquire locks, don't release
- Shrinking phase: Release locks, don't acquire
- Guarantees conflict serializability!

Deadlock:

- Transactions waiting for each other in a cycle
- Detect: Build wait-for graph, check for cycles
- Resolve: Abort victim transaction to break cycle

Quick Reference - Common Patterns

Identifying Conflicts:

Op1 / Op2	Same Item?	Conflict?
read / read	Yes	No
read / write	Yes	Yes
write / read	Yes	Yes
write / write	Yes	Yes
Any / Any	No	No

Testing Conflict Serializability:

1. Draw precedence graph (nodes = transactions)
2. Add edges for conflicts (earlier → later)
3. Check for cycles
4. No cycles = serializable

Testing 2PL:

1. Find first unlock operation
2. Check if any lock comes after it

3. If yes → NOT 2PL
4. If no → IS 2PL

Detecting Deadlock:

1. Draw wait-for graph (nodes = transactions)
2. Add edge Ti → Tj if Ti waits for Tj
3. Check for cycles
4. Cycle exists = deadlock

Common Mistakes to Avoid

Conflict Serializability:

- Don't forget: operations must be from DIFFERENT transactions to conflict
- Don't forget: read-read is NOT a conflict (both can read simultaneously)
- Don't forget: operations on different data items don't conflict
- Remember: edge direction matters! (earlier operation → later operation)

Two-Phase Locking:

- Don't confuse phases: Growing = acquire, Shrinking = release
- Critical rule: ONCE you release ANY lock, you can NEVER acquire another
- Don't forget: Check ALL lock operations, not just the first few

Deadlock:

- Wait-for graph shows "who waits for whom", not "who has what"
- Edge Ti → Tj means "Ti waits for Tj", not "Ti comes before Tj"
- Must have a CYCLE for deadlock, not just any edges

Study Strategy

For Exams:

1. Memorize ACID properties and what each means
2. Practice drawing precedence graphs (5-10 examples)
3. Practice identifying 2PL violations
4. Practice drawing wait-for graphs for deadlock detection
5. Understand WHY each concept matters (not just HOW)

Key Concepts to Master:

- ACID properties and real-world examples

- Difference between serial, concurrent, and serializable schedules
- How to test conflict serializability (precedence graph)
- Two-phase locking protocol and why it works
- Deadlock detection using wait-for graphs

Practice Makes Perfect:

- Work through examples from lecture slides
- Create your own schedules and test them
- Draw graphs for every example
- Explain concepts in plain English (to yourself or others)
- If you can teach it, you understand it!

11 Next Steps

What's Next?

This Lecture Covered:

- Transaction concept and ACID properties
- Schedules and serializability
- Conflict serializability and precedence graphs
- Lock-based concurrency control
- Two-phase locking protocol
- Deadlock detection and recovery

Next Lecture Topics:

- Recovery systems
- Understanding logging algorithms
- How atomicity and durability are implemented
- Crash recovery mechanisms

How Topics Connect:

- This lecture: How to handle CONCURRENCY (Isolation)
- Next lecture: How to handle FAILURES (Atomicity + Durability)
- Together: Complete picture of transaction management!